

```

1 # 05 TOD_Y_MATRICES_SAM
2
3 #Este archivo genera los factores TOD y las matrices de Weekday y Weekend
4
5 import pandas as pd
6 import numpy as np
7 from pathlib import Path
8 from io import StringIO
9 import sys
10
11 # 0) Leer clave del nodo desde argumentos
12 if len(sys.argv) < 2:
13     raise ValueError("Debe proporcionarse la clave del nodo como argumento (ej. 03VDR-230)")
14 clave_nodo = sys.argv[1]
15
16 # 1) Directorio base y archivo de entrada
17 base = Path(__file__).parent
18 input_csv = base / "04_matriz_horas_meses.csv"
19
20 # 2) Leer todo el archivo como texto
21 with open(input_csv, encoding="utf-8") as f:
22     lineas = f.readlines()
23
24 def extraer_bloque(nombre_bloque, n_filas=13):
25     """Devuelve un DataFrame extraído a partir del encabezado del bloque"""
26     inicio = None
27     for i, linea in enumerate(lineas):
28         if nombre_bloque in linea:
29             inicio = i + 1 # línea siguiente al título del bloque
30             break
31     if inicio is None:
32         raise ValueError(f"No se encontró el bloque '{nombre_bloque}'.")
33     bloque_csv = "".join(lineas[inicio:inicio + n_filas])
34     return pd.read_csv(StringIO(bloque_csv), index_col=0)
35
36 # 3) Matriz 'Todos los días'
37 df_todos = extraer_bloque("Matriz Todos los días")
38 valores = pd.to_numeric(df_todos.values.flatten(), errors='coerce')
39 valores = valores[~np.isnan(valores)]
40
41 if len(valores) == 0:
42     raise ValueError("No se encontraron datos numéricos válidos en 'Todos los días'.")
43
44 # 4) Histograma TOD
45 clases = 9
46 minimo = valores.min()
47 maximo = valores.max()
48 bins = np.linspace(minimo, maximo, clases + 1)
49
50 frecuencia, limites = np.histogram(valores, bins=bins)
51 frecuencia_relativa = frecuencia / frecuencia.sum()
52 frecuencia_acumulada = np.cumsum(frecuencia)
53 frecuencia_rel_acumulada = np.cumsum(frecuencia_relativa)
54 marca_clase = (limites[:-1] + limites[1:]) / 2
55 promedio_marca = marca_clase.mean()
56 tod = marca_clase / promedio_marca
57
58 # 5) Crear tabla TOD

```

```

59 tabla_frecuencias = pd.DataFrame({
60     "Límite inferior": limites[:-1],
61     "Límite superior": limites[1:],
62     "Frecuencia absoluta": frecuencia,
63     "Frecuencia relativa": frecuencia_relativa,
64     "Frecuencia acumulada": frecuencia_acumulada,
65     "Frecuencia rel. acumulada": frecuencia_rel_acumulada,
66     "Marca de clase": marca_clase,
67     "TOD": tod
68 })
69
70 # 6) Agregar fila promedio
71 fila_promedio = pd.Series({
72     "Límite inferior": np.nan,
73     "Límite superior": np.nan,
74     "Frecuencia absoluta": np.nan,
75     "Frecuencia relativa": np.nan,
76     "Frecuencia acumulada": np.nan,
77     "Frecuencia rel. acumulada": np.nan,
78     "Marca de clase": promedio_marca,
79     "TOD": np.nan
80 })
81 tabla_frecuencias = pd.concat([tabla_frecuencias, fila_promedio.to_frame().T], ignore_index=True)
82
83 # 7) Guardar TOD con nombre del nodo
84 tod_filename = base / f"TOD_{clave_nodo}.csv"
85 tabla_frecuencias.to_csv(tod_filename, index=False)
86 print(f"{tod_filename.name} generado.")
87
88 # 8) Función para clasificar matriz y exportar
89 def codificar_matriz(df_matriz, bins, nombre_base):
90     vals = pd.to_numeric(df_matriz.values.flatten(), errors='coerce')
91     indices = np.digitize(vals, bins=bins, right=False)
92     indices[indices > clases] = clases
93     indices[vals < bins[0]] = 1
94     codificada = indices.reshape(df_matriz.shape)
95     output_file = base / f"{nombre_base}_{clave_nodo}.csv"
96     pd.DataFrame(codificada, index=df_matriz.index, columns=df_matriz.columns)\
97         .to_csv(output_file, index=False, header=False)
98     print(f"{output_file.name} generado.")
99
100 # 9) Matrices Lunes-Viernes y Sábados-Domingos
101 df_weekday = extraer_bloque("Matriz Lunes-Viernes")
102 df_weekend = extraer_bloque("Matriz Sábados-Domingos")
103
104 codificar_matriz(df_weekday, bins, "Matriz_Weekday")
105 codificar_matriz(df_weekend, bins, "Matriz_Weekend")

```